Tensor Data Interchange: What Existing Solutions Cannot Express

A review of the formats, protocols, and transports that move tensor data in AI/ML inference, the gaps they leave, and a proposal for closing them.

September 2026

Abstract

AI/ML systems move large amounts of tensor data between processes, machines, and storage tiers. Apache Arrow solved the equivalent problem for tabular data, using a public specification, a self-describing schema, zero-copy buffers at a stated alignment, a streaming form, a file form, and a language-agnostic ABI (application binary interface). This review asks whether the same approach can be applied to the tensor model, where the memory layout cannot be fixed in advance and where quantization and device placement have no tabular counterpart. It examines the formats, protocols, and transports in use today and shows that each one excels in a specific area but lacks support for the others. DLPack shares memory inside one process but describes only strides. GGUF stores quantization parameters well, but only for one runtime and only in files. NIXL, NCCL, and UCX move accelerator memory across a network at full hardware speed, but they transfer byte ranges with no description attached. The consequence is visible in disaggregated large-language-model inference, where the key-value cache is transferred for every request: all production systems examined here fix shape, element type, layout, and quantization outside the transfer, send only opaque blocks and identifiers, and require hand-written conversion code whenever the two endpoints differ. From this evidence the review identifies seven gaps and states the capability each one requires: zero-copy sharing with a stated alignment, self-delimiting streaming, self-description, layout negotiation, device and memory-placement description, quantization metadata in the descriptor, and a language-agnostic ABI (application binary interface). It then states what a tensor descriptor must carry to provide them, and introduces **Hurray**, a proposed specification for a tensor interchange format designed to close all seven (github.com/pgillet/hurray, pgillet.github.io/hurray).

---

1. Introduction

Two kinds of software appear in this review, and the distinction matters only because one depends on the other.

- **Data interchange solutions** move or store tensor data. DLPack, Apache Arrow, Arrow Flight, SafeTensors, GGUF, Zarr, NetCDF, OPeNDAP, NIXL, NCCL, and UCX are in this group.
- **Compute frameworks and libraries** perform computation on tensors. PyTorch, TensorFlow, JAX, NumPy, Eigen, xtensor, PLASMA, SLATE, TVM, MLC-LLM, MLX, vLLM, and NVIDIA Dynamo are in this group. They are the clients of the first group.

When an interchange solution cannot express something the client needs, the client pays for it — with a copy, with private convention, or with adapter code written once per pair of endpoints. Each of those workarounds is evidence of a missing capability, and this review collects them.

One term is used throughout. A **descriptor** is the metadata that says how to interpret a tensor's bytes: its shape, its element type, how it is arranged in memory, how it is quantized, and where it resides. The central finding is that descriptors are rarely transmitted, and that reconstructing them out of band costs time and bandwidth.

---

2. Background: Apache Arrow and the Tabular Model

2.1 What Arrow established

Apache Arrow [2] is the starting point for this review. It solved, for one data model, a problem that remains unsolved for another, and it did so through a set of properties that the rest of this document uses as its measure. Each is stated here with the term it introduces.

- **Specification first.** The format is defined by a public specification rather than by a reference implementation, so independent implementations in many languages interoperate instead of imitating one another.
- **Zero-copy with a stated alignment.** *Zero-copy* means sharing data between components without duplicating it, by passing a pointer or a memory handle rather than the bytes. It requires agreement on alignment, ownership, and lifetime in advance. *Alignment* is the requirement that a buffer start at an address that is a multiple of some size; Arrow fixes a 64-byte minimum in the specification itself, because vector instructions (SIMD, one instruction applied to several values at once) and direct device transfers (DMA, a device reading or writing memory without the processor) reach full rate only on aligned buffers.
- **A language-agnostic ABI.** An *ABI* (application binary interface) is a fixed binary representation of structures and calls that separately compiled components can rely on. Arrow's C data interface lets two libraries hand each other a buffer without agreeing at source level or sharing a runtime.
- **A streaming format.** A stream of typed messages in which the schema precedes the data and each message is self-delimiting, so a reader can begin work before the input ends and a writer can emit batches one at a time. The same messages serve *IPC* — inter-process communication, the mechanisms by which separate processes exchange data, such as shared memory.
- **A file format.** The same data at rest, read by *mmap* — memory mapping, in which a file is placed in a process's address space so that reading it does not copy it — so storage and runtime share one representation.
- **A transport.** Arrow Flight [3] carries those same messages over a network as a streaming RPC.

Size is why the first two properties matter in practice. A single weight matrix in a 70-billion-parameter model is roughly 448 MB in 16-bit floating point, and a long-context attention cache is several gigabytes per request. A copy imposed by an unstated alignment rule costs bandwidth that the computation needs, and doubles peak memory use at the point where accelerator memory is scarcest.

2.2 Two data models, and the question this review asks

Arrow was designed for the **tabular model**: data as rows of records (typically from a relational database), each row a set of named, typed fields, stored column by column so that each column is one flat buffer of one type. The operations that model serves are filter, join, group, and aggregate. Its layout question has essentially one answer — a column is a contiguous array with a validity bitmap beside it — which is why Arrow can fix the layout in the specification and still serve every consumer.

The **tensor model** is not the same problem. A tensor is a single multi-dimensional array of one element type, addressed by an index tuple, and the operations it serves are matrix multiplication, convolution, and reduction. Its layout question has many answers, because there are many ways to map N dimensions onto linear memory and the fastest one depends on the operation and the hardware. A tensor format therefore cannot fix the layout the way Arrow does. It has to describe whichever layout the producer already holds.

The question this review asks is whether Arrow's approach can be applied to the tensor model — one public specification, one self-describing descriptor, zero-copy buffers with a stated alignment, a streaming form, a file form, and a language-agnostic ABI — given that the layout cannot be fixed, and given two further properties that have no counterpart in the tabular case. Section 3 states those three tensor-specific concerns; §§ 4 to 6 establish what existing solutions do and do not provide; § 7 states the capabilities that are missing; § 8 states the descriptor they imply; and § 9 introduces Hurray, a proposal to specify it.

3. What a Tensor Adds

Three things a tensor must state have no equivalent in the tabular model, and a consumer that does not know all three cannot use the bytes. Section 4 shows which existing solutions can state them.

Layout. A layout is how a tensor's elements are arranged in memory. The simplest description is *strided*: one step size per dimension, which covers row-major order, column-major order, transposes, and slices. Fast kernels do not use it. They use *tiled* layouts, which store small rectangular blocks contiguously so that each block fits in cache, and *packed* layouts, which rearrange operands into the exact order a vector or matrix unit reads them. A *dense* tensor stores every element explicitly (no implicit zeros), while a *sparse* tensor stores only non-zero elements along with an index structure (for example, compressed sparse rows). Attention caches use *paged* layouts, described in § 5. No single layout is universally optimal. The best choice depends on the operation, the hardware, and where the memory hierarchy bottlenecks.

Quantization. Inference stores weights and caches at reduced precision because their size is the binding constraint: a 70-billion-parameter model is about 140 GB in 16-bit floating point and about 35 GB at 4 bits, and decode is bound by memory bandwidth (§ 5), so reading fewer bytes per parameter directly raises throughput. Quantization stores a value as a low-precision integer together with a scale and, optionally, a zero point, so that the value is approximated by `scale × (quantized − zero_point)`. Those parameters may apply to the whole tensor, to one channel, or to a group of consecutive elements, typically 32 or 64. Where elements are narrower than a byte, several share a byte in an order that has to be stated, because more than one convention is in use. None of this is recoverable from the bytes: a tensor that loses its layout can still be read, incorrectly, whereas a tensor that loses its quantization parameters cannot be read at all. That is why they have to travel with it.

Device and memory placement. A buffer lives in *host memory* (the system RAM the CPU addresses), in *device memory* (an accelerator's own memory, such as a discrete GPU's, which the CPU cannot read directly), in *unified memory* (one physical pool that both address, as on Apple Silicon), or in a *registered* region pinned and published to a network interface so a remote machine can read or write it. A consumer cannot use a buffer it cannot locate.

4. The Landscape

4.1 Interchange solutions

Solution	Kind	Layout model	Quantization	Streaming	Zero-copy	RDMA	Self-describing	Adoption
DLPack [1]	In-process ABI	Strided	✗	✗	✓	✗	Partial	Very high
Apache Arrow [2]	IPC, columnar	Row/column-major	✗	✓	✓	✗	Partial	Very high
Arrow Flight [3]	Streaming RPC	Row/column-major	✗	✓	✗	✗	Partial	Medium
SafeTensors [4]	File	Row-major	✗	✗	✓ (mmap)	✗	Partial	High
GGUF [5]	File	Row-major + packed blocks	✓ informal	✗	✓ (mmap)	✗	✓	High
Zarr [6]	File, object store	Chunk grid	✗	✗	✗ (compressed)	✗	✓	Medium
NetCDF [7]	File	Row-major	✗	✗	✗	✗	✓	High

Solution	Kind	Layout model	Quantization	Streaming	Zero-copy	RDMA	Self-describing	Adoption
OPeNDAP [8]	HTTP request	Row-major	✗	✓	✗	✗	✓	Medium
NIXL [9]	RDMA transport	None	✗	n/a	✓	✓	✗	Emerging
NCCL [10]	RDMA collectives	None	✗	n/a	✓	✓	✗	Very high
UCX [11]	RDMA abstraction	None	✗	n/a	✓	✓	✗	High

Table 1: Data interchange solutions. *Self-describing* means the shape, element type, layout, and quantization travel with the data.

Five facts from this table drive the rest of the review.

1. **No solution describes more than one layout family.** Every entry is limited to strides or to row-major order. None can state that a tensor is tiled, packed, sparse, or paged.
2. **Only GGUF puts quantization parameters in the descriptor**, and its schemes are defined by their reference implementation rather than by a portable specification, so interoperability requires reading that code.
3. **Arrow Flight loses the alignment Arrow specifies.** It carries data over gRPC, which requires at least one CPU copy per message and does not preserve alignment, so receivers copy again before handing memory to an accelerator or a linear-algebra kernel. Its message structure is nonetheless the right one: the descriptor precedes the data, messages are typed, and exchange is bidirectional.
4. **The RDMA transports describe nothing by design.** RDMA is remote direct memory access: one machine's network interface reads or writes another machine's registered memory without involving the remote processor. NIXL, NCCL, and UCX move registered byte ranges and require both endpoints to already agree on the format.
5. **File formats stop at the file.** SafeTensors, GGUF, Zarr, and NetCDF have no in-process ABI and no streaming protocol, so none of them can serve runtime interchange.

4.2 Compute frameworks and libraries

Framework	Interchange it uses	What it cannot express at the boundary
NumPy [12]	DLPack, buffer protocol, `.npy`	Nothing beyond strides; no path outside Python
PyTorch [13]	DLPack, SafeTensors, NCCL, RDMA libraries via serving stacks	Quantization parameters (kept in separate objects); packed and tiled layouts
TensorFlow [14]	DLPack, saved-model container, NCCL	Compiler-chosen physical layout; quantization is a property of the model artifact
JAX [15]	DLPack, checkpoint libraries, NCCL	Device sharding and compiler-chosen tiling; a handoff degrades to a dense single-device view
Eigen [16], xtensor [17]	None; both map caller-owned memory	Any layout not fixed at compile time; no descriptor of any kind
PLASMA [18], SLATE [19]	None	Tile parameters, which are private to the library although central to its performance
TVM [20], MLC-LLM [21]	DLPack; ad-hoc parameter bundles	The packed and tiled forms the compiler produces; grouped low-bit weight parameters
MLX [22]	DLPack, buffer protocol, existing file formats	Its packed quantized representation; unified memory has no single owning device
vLLM [23]	NIXL, external cache layers, NCCL	Paged cache geometry and quantization; fixed once at startup

Framework	Interchange it uses	What it cannot express at the boundary
NVIDIA Dynamo, TensorRT-LLM [24], [25]	NIXL, UCX, MPI	Cache layout across mismatched parallelism, handled by a hand-written module

Table 2: What the clients use, and what they cannot state at the boundary.

Three observations follow. First, nine of these systems support DLPack, so its descriptor sets the effective limit on what can cross a boundary inside one process. Second, the numerical libraries prove that adopting foreign memory is routine — Eigen and xtensor both map caller-owned buffers — so the obstacle is the missing description, not the sharing mechanism. Third, PLASMA and SLATE show that a serious library maintains several layouts at once, which means a single mandated layout would force a conversion on someone in every exchange.

---

5. Where the Gap Is Most Expensive

Large-language-model inference has two phases. **Prefill** processes the whole prompt at once, is compute-bound, and fills the **key-value (KV) cache**: the stored attention keys and values that let later steps avoid recomputing attention over the prompt. **Decode** emits one token at a time, is memory-bandwidth-bound, and reads and extends that cache. The two phases have different hardware profiles, so production systems run them on separate accelerators or nodes [26], [27]. The cache, of logical shape `[layers, 2, heads, seq_len, head_dim]`, must then be transferred for every request — several gigabytes at long context lengths. The cache is stored in a **paged** layout [23]: a pool of fixed-size blocks plus a per-sequence block table mapping logical positions to physical blocks, so a transfer moves a list of non-contiguous blocks rather than one contiguous region.

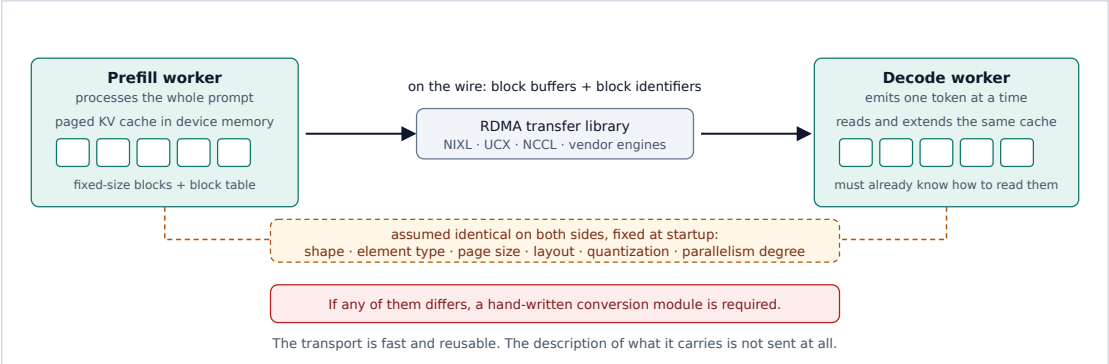

Figure 2. KV cache transfer between a prefill worker and a decode worker.

System	Transport used	Sent with the data	Assumed out of band
DistServe [26]	Intra-node interconnect	Layer and block references	Identical model build and cache layout
Mooncake [27]	Its own multi-NIC RDMA engine	Block keys and offsets	Shape, element type, paged layout
vLLM connectors [23], [28]	NIXL, Mooncake, LMCache	Raw blocks and block identifiers	Everything, fixed at cache-registration time
Dynamo, TensorRT-LLM [24], [25]	NIXL, UCX, MPI	Prompt tokens and connection parameters	Layout; parallelism mismatch resolved by a bespoke module
llm-d [29]	NIXL with a unified collective backend	Blocks and routing hints	Model identity and layout

System	Transport used	Sent with the data	Assumed out of band
LMCache [30], [31]	Engine connectors, multi-tier store	Compressed chunks and keys	Cache format and compression codec

Table 3: Six systems that transfer the KV cache.

The pattern is identical in all six. The handshakes negotiate addresses and agent identity; none of them negotiates a descriptor. Three costs follow.

1. **Endpoints are coupled in pairs.** Every connector assumes matching builds on both sides. Transfer between different engines, different versions, or different quantization settings requires a purpose-built adapter, because there is no neutral representation.
2. **Layout conversion is written by hand.** When prefill and decode run at different tensor-parallelism degrees, the block mapping differs; TensorRT-LLM converts the layout during transmission in a dedicated module, and vLLM's RDMA connector reshuffles the block mapping itself. A description of the source and destination layouts would let one routine handle every such case.
3. **Stored caches are unreadable elsewhere.** A cache written to a pool [27] or compressed to disk [31] carries no standard description, so only the software that wrote it can read it back. This removes most of the benefit of pooling it.

---

6. A Second Gap: Heterogeneous Composition

Every layout in § 4 describes one element type, one layout, and one quantization scheme across the whole tensor. Several established techniques do not fit that model: one logical tensor is assembled from regions that differ in precision, in layout, or in both, and each region has its own buffers. Two composition rules appear in practice, and they answer different questions about what a position in the tensor means.

Partition: regions that do not overlap. Each position belongs to exactly one region. A residual-precision KV cache [34] is the clearest example. The most recent tokens are kept at full precision, because they are still being written and are the most sensitive to quantization error, while older tokens are stored 2-bit quantized. The split runs along the sequence axis; the two regions have different element types, different quantization parameters, and separate buffers; and no position is in both. A mixture-of-experts model that assigns a different bit width per expert partitions the same way, along the expert axis. Outside machine learning the pattern is long established and production-proven: adaptive-mesh frameworks store one logical array as independently allocated boxes at different refinement levels [35], volumetric formats mix constant tiles with dense leaves [36], and HDF5 virtual datasets define one logical dataset as per-region mappings onto separate source files [37].

Overlay: corrections on top of a base. A base tensor spans the whole index space, and a sparse second tensor supplies replacement values at scattered positions that the base also covers. SpQR [32] uses this arrangement for sparse-quantized weights: it stores a weight matrix at 3–4 bits per weight and keeps roughly one percent of the weights — the outliers whose quantization error dominates the loss — at higher precision in a separate sparse structure. KVQuant [33] applies the same arrangement to the KV cache. Reading position `(i, j)` means consulting the sparse structure first and falling back to the dequantized base only if no correction is stored there. Because base and corrections share positions, this cannot be expressed as a partition.

What this requires of a descriptor. The same set of regions has two different meanings under the two rules: under a partition the regions are the tensor, and under an overlay all but one of them are exceptions to it. A descriptor must therefore carry three things — the geometry of the regions, a complete description of each region (element type, layout, quantization, buffers), and the composition rule that resolves a position. No mainstream tensor interchange format carries any of the three, so both arrangements are today private to the library that implements them.

7. The Gaps

Four of the seven gaps below — alignment, streaming, a transmitted descriptor, and a language-agnostic ABI — are properties Arrow already provides for tabular data and no tensor solution provides. The other three are the tensor-specific concerns of § 3: layout, device placement, and quantization.

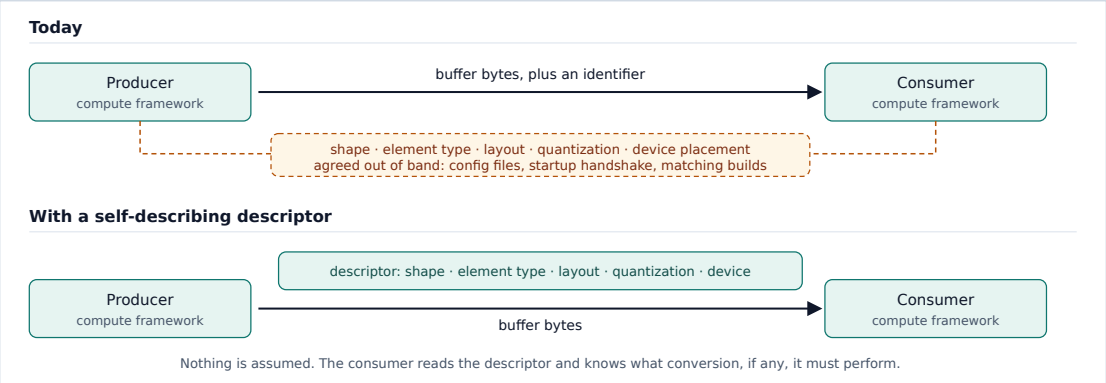

Figure 1. What crosses the boundary today, and what a self-describing descriptor changes.

#	Gap	What happens today	Required capability
1	No stated alignment	Receivers copy defensively before using a buffer; Arrow Flight loses alignment through gRPC	A normative minimum alignment, stricter where accelerator and IPC paths need page alignment, plus explicit lifetime transfer
2	No streaming form	File formats load whole artifacts; readers buffer input they cannot yet use	Descriptor before data, self-delimiting frames, no trailing index or back-reference in the stream
3	No descriptor on the wire	Format is fixed at startup and endpoints must match (§ 5)	A descriptor sent with every transfer: shape, element type, layout, quantization, device, and position within a larger tensor
4	One layout family per format	Producers repack, or endpoints agree privately; mismatches are resolved by hand-written modules	A layout vocabulary covering strided, tiled, sparse, paged, and composite forms, the composition rule for the last of these (§ 6), an extension path for hardware-specific packings, and negotiation so conversion happens once on the better-placed side
5	No device placement	Placement lives in engine configuration; unified-memory systems have no single owning device	A placement model covering host, discrete, unified, and registered memory, in which device affinity can belong to an access rather than to the buffer
6	No quantization metadata	Parameters travel in config files or framework-private objects; sub-byte packing order differs between implementations	Scheme identifier, scales, zero points, and block size in the descriptor, with bit-exact packing order (which bits hold which sub-byte element, § 3) and a normative, versioned scheme set
7	No language-agnostic ABI	Formats stop at a file boundary or at one language's ecosystem	A stable C ABI carrying the descriptor and buffer handles, with no idioms of the implementation language

Table 4: Seven gaps, what they cost today, and the capability each requires.

No solution in Table 1 addresses more than three of the seven. DLPack addresses 1 and 7 inside one process. Arrow addresses 1, 2, and 7 for a tabular data model. GGUF addresses 3 and 6 for one runtime, in files. The RDMA transports address none of them, which is correct for their role: they are a data plane, and what is missing is a description that travels above them.

8. What the Descriptor Must Carry

Table 4 states seven capabilities. Six of them are properties of a single artifact that does not exist today: a descriptor attached to every tensor, on every path it travels. Collecting what the preceding sections require, that descriptor must carry:

- **Shape and element type**, including the sub-byte types quantization produces.
- **Layout**: which family the tensor uses — strided, tiled, sparse, paged, or composite — together with that family's parameters: strides, tile shape, index buffers, page size and block table, or the member list and composition rule of § 6.
- **Quantization**: scheme identifier, scales, zero points, group size, and the packing order of § 3, so that a consumer can dequantize without external configuration.
- **Device placement**: which of the four locations of § 3 each buffer occupies.
- **Buffer geometry**: the offset, length, and alignment of each buffer, with ownership and lifetime stated, so that a consumer can hold the memory instead of copying it.
- **Position within a larger tensor**: the offset and extent of this tensor inside the whole, which is what a sharded handoff loses today (§ 4.2).

The seventh capability, negotiation, is not a field. It is what two endpoints do with these descriptors before any data moves: each declares what it can consume, and the conversion, if one is needed, is performed once by the side better placed to perform it.

---

9. Hurray: A Proposal

Hurray is a proposed specification for a tensor interchange format and protocol, designed against the seven gaps above. It specifies the descriptor of § 8, a binary encoding for it, and the protocol properties Table 4 requires around it: a normative minimum buffer alignment with explicit ownership transfer, a self-delimiting stream in which each descriptor precedes its data and nothing refers backwards, a capability handshake in which each side declares the layouts, element types, and quantization schemes it can consume, and a stable C ABI that any language can implement against. The same descriptor is carried on every path — in-process handoff, IPC stream, file container, and RDMA data plane — so a tensor does not change form when it changes route. Hurray defines no kernels, no scheduler, and no cache policy: compute frameworks remain the clients, and existing transports remain the data plane.

What would be new is the combination. The individual capabilities all exist somewhere. DLPack shares memory inside one process but describes only strides. Arrow supplies the buffer and IPC discipline, for a tabular data model. GGUF puts quantization parameters in the container, for one runtime and only in files. NIXL and UCX move accelerator memory across a network without describing what they move. No solution in Table 1 provides more than three of the seven capabilities, and none combines these four:

- one layout vocabulary spanning strided, tiled, sparse, paged, and composite tensors;
- quantization metadata inside the descriptor rather than beside it;
- a self-describing tensor carried over an RDMA data plane;
- a language-agnostic C ABI, so the format can be implemented in any language rather than bound from one.

The costs are real. Three objections apply to any format of this kind, and a fourth applies to this one.

- **Complexity.** Every named layout is a burden on every implementation: a reader handling strided, tiled, sparse, paged, and composite tensors is larger and harder to verify than one handling strided layouts only. Two things bound the cost. The mandatory core can stay small, with the rest optional and negotiated. And an implementation that meets an unsupported layout need only reject it explicitly, which is far cheaper than supporting it and is enough to keep the format safe to extend.
- **Negotiation cost.** A handshake adds round trips before the first byte moves, and for a single small tensor that is pure overhead. But it is paid once per session, while the conversion it avoids is paid per

transfer and grows with the size of the data: at the buffer sizes of § 5, one avoided repack pays for many handshakes. Where it does not pay, the handshake must be skippable, which is a requirement on the design rather than an argument against it.

- **Adoption.** A new ABI needs framework support, and DLPack already has it almost everywhere. This is the largest risk, and no analysis removes it. Two things reduce it. DLPack itself shows that an ABI spreads when it solves a problem frameworks actually have. And the two are not exclusive: a dense strided tensor crosses either boundary, so a framework can adopt the richer descriptor only where DLPack cannot express what it needs.
- **Why not extend an existing format?** This is the cheapest option, and it was considered. DLPack's structure is deliberately minimal and fixed; adding layout, quantization, device, and shard fields to it produces a different artifact with the same name, and it would still have no streaming or file form. Arrow's data model is tabular, and its tensor extension inherits that. GGUF is a file format with no ABI and no streaming protocol. Each would have to acquire what the other two have, which is a larger change than specifying the descriptor once and mapping it onto all three.

Hurray is a standardization effort, and the specification is public. The format is documented at **pgillet.github.io/hurray** and developed in the open at **github.com/pgillet/hurray**. Review of the specification, and of the gap analysis in § 7 that motivates it, is welcome.

Several questions remain open and are stated as such. How large should the layout vocabulary be, given that every named layout is a burden on every implementation and every omission forces a copy? What should negotiation exchange beyond a list of supported layouts, given that the better-placed side depends on conversion cost that neither side currently expresses? Should device affinity belong to a buffer or to an access? How should quantization schemes be parameterized so that new ones do not require a new scheme identifier each time?

---

10. Conclusion

The transports used for tensor data are fast, general, and widely deployed. The descriptions of what they carry are not transmitted at all. Compute frameworks compensate individually: PyTorch keeps quantization parameters outside the tensor, JAX cannot express sharding across a handoff, vLLM fixes its cache format at startup and sends opaque blocks, and TensorRT-LLM contains a hand-written layout converter. These are not defects in those systems; they are what an interchange layer with an insufficient descriptor forces on their authors.

What is missing is a portable description that travels with the bytes, expressive enough to state that a tensor is tiled, paged, sparse, block-quantized, or composed of heterogeneous regions, and carried identically across in-process, IPC, file, and RDMA paths. Table 4 states the capabilities such a description requires and § 8 states its contents. Hurray is a proposal to specify both; § 9 states what that would provide and what it would cost.

References

[1] DLPack: Open In-Memory Tensor Structure. https://github.com/dmlc/dlpack
[2] Apache Arrow. https://arrow.apache.org
[3] Apache Arrow Flight. https://arrow.apache.org/docs/format/Flight.html
[4] SafeTensors. Hugging Face. https://github.com/huggingface/safetensors
[5] GGUF: GPT-Generated Unified Format. https://github.com/ggerganov/ggml/blob/master/docs/gguf.md
[6] Zarr: chunked, compressed, N-dimensional arrays. https://zarr.dev
[7] NetCDF. Unidata / UCAR. https://www.unidata.ucar.edu/software/netcdf/
[8] OPeNDAP (DAP2 / DAP4). https://www.opendap.org
[9] NIXL: NVIDIA Inference Xfer Library. https://github.com/ai-dynamo/nixl
[10] NCCL: NVIDIA Collective Communications Library. https://developer.nvidia.com/nccl

[11] P. Shamis et al., "UCX: An Open Source Framework for HPC Network APIs and Beyond," *HOTI*, 2015. https://openucx.org
[12] C. R. Harris et al., "Array programming with NumPy," *Nature* 585, 357–362, 2020.
[13] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *NeurIPS*, 2019. https://arxiv.org/abs/1912.01703
[14] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," *OSDI*, 2016. https://arxiv.org/abs/1605.08695
[15] JAX. https://docs.jax.dev
[16] Eigen. https://eigen.tuxfamily.org
[17] xtensor. https://xtensor.readthedocs.io
[18] PLASMA: Parallel Linear Algebra Software for Multicore Architectures. https://icl.utk.edu/plasma/
[19] M. Gates et al., "SLATE: Software for Linear Algebra Targeting Exascale," SLATE Working Notes. https://icl.utk.edu/slate/
[20] T. Chen et al., "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," *OSDI*, 2018. https://arxiv.org/abs/1802.04799
[21] MLC-LLM. https://llm.mlc.ai
[22] MLX: an array framework for Apple silicon. https://ml-explore.github.io/mlx/
[23] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," *SOSP*, 2023. https://arxiv.org/abs/2309.06180
[24] NVIDIA Dynamo. https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models/
[25] Disaggregated Serving in TensorRT-LLM. NVIDIA. https://nvidia.github.io/TensorRT-LLM/blogs/tech_blog/blog5_Disaggregated_Serving_in_TensorRT-LLM.html
[26] Y. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," *OSDI*, 2024. https://arxiv.org/abs/2401.09670
[27] R. Qin et al., "Mooncake: A KVCache-centric Architecture for Serving LLM Chatbot," *USENIX FAST*, 2025. https://arxiv.org/abs/2407.00079
[28] vLLM: disaggregated prefilling and KV cache connectors. https://docs.vllm.ai/en/stable/features/disagg_prefill/
[29] llm-d: Kubernetes-native distributed inference. https://llm-d.ai/docs/guide/Installation/pd-disaggregation
[30] LMCache: a KV cache layer for LLM serving. https://arxiv.org/html/2510.09665v2
[31] Y. Liu et al., "CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving," *ACM SIGCOMM*, 2024. https://arxiv.org/abs/2310.07240
[32] T. Dettmers et al., "SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression," 2023. https://arxiv.org/abs/2306.03078
[33] C. Hooper et al., "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," 2024. https://arxiv.org/abs/2401.18079
[34] Z. Liu et al., "KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache," 2024. https://arxiv.org/abs/2402.02750
[35] W. Zhang et al., "AMReX: A Framework for Block-Structured Adaptive Mesh Refinement," *JOSS*, 2019. https://amrex-codes.github.io
[36] K. Museth, "VDB: High-Resolution Sparse Volumes with Dynamic Topology," *ACM TOG*, 2013. https://www.openvdb.org
[37] HDF5 Virtual Datasets. The HDF Group. https://docs.hdfgroup.org/hdf5/develop/_v_d_s.html